

Performance Tuning and Monitoring

Part 4



Tune memory requirements, learn about changing kernel behaviour in relation to memory allocation, fiddle with the memory overcommit feature, and enhance performance by tuning the ARP cache.

Last month we learned how to tune the performance of our hard disks and RAID arrays. Performance tuning is always incomplete without tuning the memory. Therefore, in this article we will learn how to tune the memory to enhance the performance of the system using TLB, strace, demand paging, memory overcommit, and by tuning the ARP cache.

Memory addressing and performance

For performance reasons, the memory on a computer is organised into fixed-sized blocks, or chunks, known as pages. The size of the pages varies from processor to processor. On a normal x86 system, the default page size is 4 KB (4,096 bytes).

The RAM on the computer is divided into page-frames. One page frame of RAM can hold one page of data. Whenever a process tries to access a particular memory location it must be directed to the appropriate page-frame in the memory (RAM). In case that information is not available in the page-

frame (RAM), then the kernel must find that information and load it into the page-frame.

Each process has its own page table. Each page table entry (PTE) contains information about one page-frame that has been assigned to the process.

The Linux kernel also reserves some memory at boot time for internal functions. You can use the `dmesg | grep Memory` command to view the amount of reserved memory.

```
[root@legacy ~]# dmesg | grep Memory
[ 0.000000] Memory: 3351360k/3406668k available
(3264k kernel code, 54032k reserved, 2046k data, 404k
init, 2501460k highmem)
```

You can also verify your page size by running the `getconf -a | grep SIZE` command. Shown below is the output from my machine:

```
[root@legacy ~]# getconf -a | grep SIZE
PAGESIZE 4096
PAGE_SIZE 4096
<<<<output truncated>>>>
```